

BUSINESSAI

TECHNICAL ASSESSMENT

Backend Engineer - Take-Home Project

Python | SQL | Flask / FastAPI / Django | Up to 7 Days | 100 Points

(Or your choice of Language and Framework(s))

The Brief

You will build a Bookmarks API: a small but complete backend service that lets users save, tag, search, and manage web bookmarks. You have up to 5 days to complete it at your own pace. The project requires zero paid services - everything runs locally with free, open-source tools. We estimate 4-6 hours of actual coding time spread across the week however you prefer.

The Product

A personal bookmarks manager exposed as a RESTful JSON API.

Core Features

- CRUD for bookmarks (URL, title, description).
- Tag bookmarks with one or more tags.
- Filter & search by tag, title keyword, and date range.

Users & Auth

- Simple user registration & login. JWT-based auth.
- Users can only see and manage their own bookmarks.

Data Layer

- Any free database: SQLite (recommended - zero setup, built into Python), PostgreSQL, or MySQL running locally. No paid cloud databases.
- Proper schema with migrations. At least 3 tables: users, bookmarks, tags (many-to-many).

API Documentation

- OpenAPI/Swagger spec served at /docs or /api/docs.
- All endpoints documented with request/response schemas, status codes, and auth requirements.

Quality

- Input validation. Error handling with consistent JSON responses.
- At least 10 tests covering happy paths, edge cases, auth, and OpenAPI contract validation.

Zero Cost To You

This assessment requires only free, open-source tools. No cloud accounts, no paid APIs, no Docker required. Python, a local database, and pip are all you need.

Deliverables

Submit a link to your Git repository (preferably GitHub)

Rules & Expectations

FRAMEWORK:

Use Flask, FastAPI, or Django (DRF) (your choice). Also happy to consider other framework(s).

All are free and open source.

We want to see you work in a framework you're comfortable with.

TIMELINE:

7 calendar days from when you receive this. We estimate 4-6 hours of actual work - pace it however suits your schedule. Partial completion with high quality beats rushed completion.

ZERO-COST GUARANTEE:

Everything runs locally: Python 3.10+, a free database (SQLite, PostgreSQL, or MySQL), pip packages. No cloud accounts, no paid APIs, no Docker required. If any step would cost you money, skip it that is not the intent.

RESOURCES:

You may reference official documentation. No AI code generation tools. We will ask you to walk through your code in a follow-up call.

BONUS POINTS:

Rate limiting, Docker setup, a seed script that populates sample data, or cursor-based pagination. Not required only if time allows and they are free to implement.

WHAT TO SUBMIT:

A Git repository (GitHub/GitLab link or zip). We review your commit history, so commit as you go rather than in one batch at the end.

Build Milestones

Work through these in order. Each milestone builds on the last and maps to a scoring category.

01 - Project Setup & Data Models

[Python] [SQL] | ~30 min

Initialise the project with your chosen framework. Set up your database connection, define your ORM models, and generate initial migrations. Use any free, locally-run database - SQLite is the easiest, but PostgreSQL or MySQL are also fine.

Required models: User, Bookmark, Tag, and a bookmark_tags association table.

```
User
- id (PK)
- username (unique, max 80 chars)
- email (unique)
- password_hash
- created_at

Bookmark
- id (PK)
- url (required, valid URL)
- title (required, max 200 chars)
- description (optional, max 500 chars)
- user_id (FK -> User)
- created_at, updated_at

Tag
- id (PK)
- name (unique, lowercase, max 50 chars)

bookmark_tags (association table)
- bookmark_id (FK), tag_id (FK)
```

WHAT WE EVALUATE:

- Correct relational modelling (many-to-many for tags)
- Appropriate column types, constraints, and indexes
- Clean project structure and dependency management
- Migrations that run without errors
- Project Architecture
- Deployment Architecture

02 - User Registration & JWT Auth

[Framework] [Python] | ~40 min

Implement user registration and login endpoints that issue JWT tokens. All subsequent bookmark endpoints must require a valid token.

```

POST /api/auth/register
Body: { "username", "email", "password" }
-> 201 { "user": { "id", "username", "email" }, "token": "..." }

POST /api/auth/login
Body: { "email", "password" }
-> 200 { "user": {...}, "token": "..." }

```

WHAT WE EVALUATE:

- Secure password handling (hashing, not encryption)
- Correct JWT implementation (signing, verification, expiry)
- Auth middleware cleanly separated from route logic
- Proper HTTP status codes and error messages

03 - Bookmark CRUD Endpoints

[Framework] [SQL] | ~45 min

Build the full CRUD for bookmarks. Every endpoint must be scoped to the authenticated user - users must never see each other's bookmarks.

```

POST /api/bookmarks
Body: {
  "url": "https://example.com/article",
  "title": "Great Article",
  "description": "An insightful read on...",
  "tags": ["python", "tutorial", "backend"]
}

```

WHAT WE EVALUATE:

- Correct RESTful conventions (verbs, status codes, resource naming)
- Robust input validation with helpful error messages
- Proper ownership scoping (no data leaks between users)
- Clean handling of the many-to-many tag relationship

04 - Search, Filter & SQL Queries

[SQL] [Python] | ~40 min

Add query parameters to the list endpoint so users can search and filter their bookmarks. Support filters: tag, q (keyword), from/to dates, and pagination.

Write a raw SQL query (not just ORM) for a stats endpoint:

```

GET /api/bookmarks/stats -> {
  "total_bookmarks": 142,
  "total_tags": 28,
  "top_tags": [{"name": "python", "count": 45}, ...],
  "bookmarks_per_month": [{"month": "2025-01", "count": 23}, ...]
}

```

WHAT WE EVALUATE:

- Efficient query construction (avoid N+1, use joins)
- Correct SQL aggregation (GROUP BY, COUNT, date functions)
- Pagination implemented correctly with total count

05 - OpenAPI/Swagger Documentation

[Framework] [Design] | ~30 min

Your API must serve a complete OpenAPI 3.0+ specification with interactive documentation accessible at /docs or /api/docs.

WHAT WE EVALUATE:

- Spec is complete and accurate
- Schemas are reusable
- Auth requirements documented

06 - Error Handling & Validation

[Framework] [Python] | ~20 min

Implement a global error handling layer that ensures every error response follows a consistent JSON structure.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Title is required and must be under 200
characters.",
    "details": { "field": "title", "constraint": "max_length",
"limit": 200 }
  }
}
```

07 - Testing (with OpenAPI Validation)

[Python] [Framework] [OpenAPI] | ~35 min

Write at least 10 tests using pytest (or Django's test runner). Include tests that validate that actual API responses conform to the OpenAPI spec schemas.